FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

MATHÉMATIQUES & INFORMATIQUE

Rapport de Projet:

Orchestration et Déploiement Continu d'une Application de Recommandation Académique

Filière : Master d'Excellence IA
Modèle : Cloud & DevOps

Réalisé par :
Benjelloun Abdderrahmane
Houssame Bouyous
Chbab Brahim
Assofi Nada

Encadré par :
Pr. Faouzia Benabbou

Table de Matières :

I. Introduction et Contexte.....3
1.1 Présentation du Projet : L'IA au service de l'orientation.....3
1.2 Objectifs DevOps : L'Infrastructure comme produit.....3
1.3 Architecture Globale : Le Cycle de Vie du Projet.....4
A. Zone 1 : Développement Local et Provisionnement.....4
B. Zone 2 : Le Moteur CI/CD (GitHub & GitHub Actions).....5
C. Zone 3 : L'Environnement de Production (Azure Spain Central).....6
1. Isolation et Sécurité Réseau.....7
2. Architecture du Cluster K3s.....7
Conclusion de l'Architecture.....8
II. Conception Logicielle et Conteneurisation.....9
2.1 Le Backend (Flask & Gunicorn).....9
2.2 Le Frontend (React & Nginx).....10
2.3 Registre d'images : Docker Hub.....12
III. Infrastructure as Code (IaC) avec Terraform.....13
3.1 Provisionnement sur Azure (main.tf).....13
3.2 Sécurité Réseau (NSG).....15
3.3 Intégration Ansible : Inventaire Automatisé.....16
IV. Configuration Automatisée avec Ansible.....17
4.1 Préparation du Runtime (Rôle common).....17
4.2 Déploiement et Initialisation du Cluster (Rôle k3s_master).....19
4.3 Jonction du Nœud de Travail (Rôle k3s_worker).....20
4.4 Extraction et Préparation du Kubeconfig.....20
V. Orchestration et Résilience du Cluster (K3s).....21
5.1 Abstraction des Workloads et Contrôle d'État.....22
5.2 Topologie Réseau et Exposition des Services.....23
A. Communication Inter-services (ClusterIP).....23
B. Routage de Couche 7 (Ingress NGINX).....24
5.3 Persistance des Données (PV/PVC).....25
VI. Automatisation CI/CD avec GitHub Actions.....27
6.1 Architecture du Workflow de Déploiement.....27
6.2 Gestion et Sécurisation des Secrets.....29
6.3 Stratégie de Tagging et Immutabilité.....29
VIII. Guide de Reproduction (Documentation).....31
8.1 Pré-requis et Préparation des Identifiants.....31

Obtention des identifiants Azure....32
Étape 1 : Génération des clés de sécurité....32
Étape 2 : Configuration des variables Terraform....33
Étape 3 : Déploiement de l'Infrastructure....33
Étape 4 : Configuration Logicielle avec Ansible....36
Étape 5 : Préparation du pilotage Kubernetes....39
8.3 Initialisation du Pipeline CI/CD (GitHub)....40
8.4 Maintenance et Destruction....41
IX. Conclusion....42
9.1 Bilan Technique et Défis Surmontés....42
9.2 Perspectives et Évolutions....42

I. Introduction et Contexte

1.1 Présentation du Projet : L'IA au service de l'orientation

Dans le système universitaire actuel (LMD), le passage de la Licence au Master est un pivot critique. Les étudiants se retrouvent souvent face à une multitude de spécialisations (Big Data, Cybersécurité, Systèmes Embarqués) sans avoir une vision claire de la correspondance entre leurs résultats académiques et les exigences de ces filières.

Notre projet répond à ce besoin par le développement d'un **Système Intelligent de Recommandation Académique**.

- **Le rôle de l'IA :** En analysant les performances de l'étudiant (modules S1 à S6), un modèle de Machine Learning (Random Forest) prédit la spécialisation la plus adaptée.
- **L'application :** Une plateforme Full-Stack permettant une saisie fluide des notes et une visualisation immédiate de l'orientation suggérée.

1.2 Objectifs DevOps : L'Infrastructure comme produit

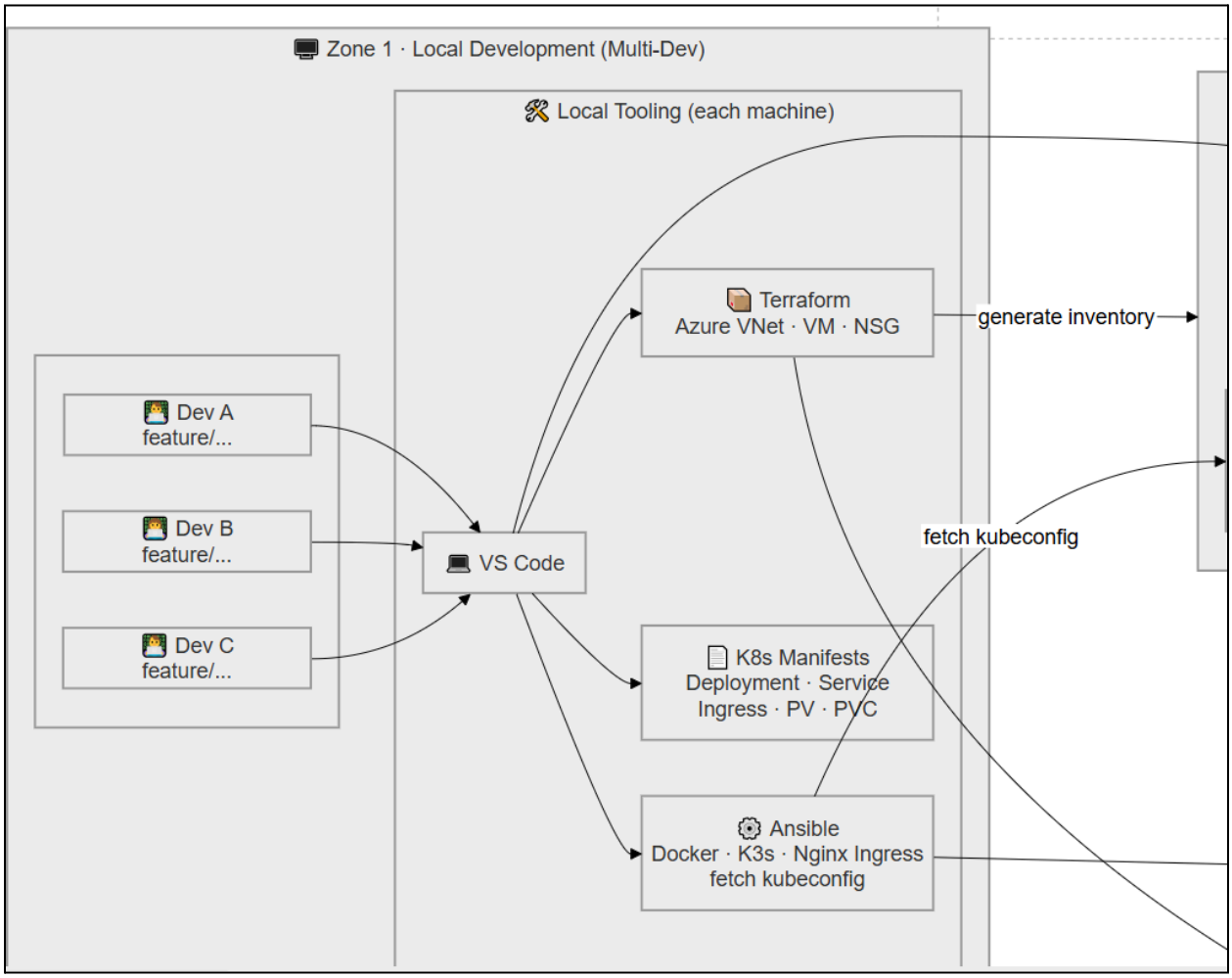
L'enjeu technique majeur n'est pas seulement l'application, mais la création d'une **plateforme de livraison continue**. L'objectif est d'éliminer les interventions manuelles ("No-Ops") via trois piliers :

1. **Automatisation via l'IaC (Infrastructure as Code) :** Utiliser Terraform pour garantir que l'infrastructure cloud en Espagne est identique à chaque déploiement.
2. **Gestion de Configuration :** Utiliser Ansible pour transformer des machines virtuelles "vierges" en un cluster Kubernetes opérationnel en quelques minutes.
3. **Haute Disponibilité :** Orchestrer les conteneurs avec K3s pour assurer une résilience des services (auto-healing) et une gestion intelligente du trafic.

1.3 Architecture Globale : Le Cycle de Vie du Projet

L'architecture du projet repose sur une segmentation en trois zones fonctionnelles interdépendantes. Cette structure garantit une séparation stricte entre l'environnement de développement, la logique d'intégration et l'infrastructure de production.

A. Zone 1 : Développement Local et Provisionnement



Le cycle de vie débute par la définition de l'infrastructure et de la configuration logicielle via le paradigme de l'**Infrastructure as Code (IaC)**. Cette étape assure la reproductibilité intégrale de l'environnement.

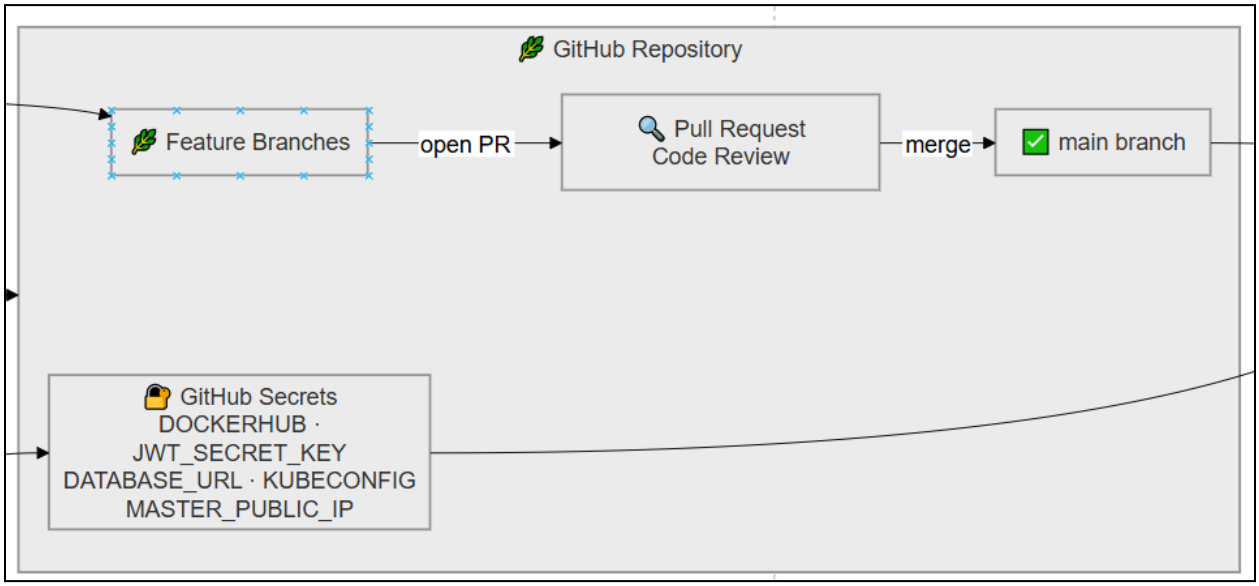
- **Terraform (Provisionnement Cloud) :** L'outil orchestre la création des ressources sur le fournisseur **Azure**. Il définit le réseau virtuel (VNet 10.0.0.0/16), le sous-réseau (10.0.1.0/24), et les groupes de sécurité réseau (NSG). Il déploie deux instances virtuelles (Master et Worker) et génère dynamiquement le fichier d'inventaire indispensable à la phase suivante.

- **Ansible (Gestion de Configuration) :** Une fois les instances actives, Ansible automatise l'installation du moteur de conteneurisation Docker et du cluster Kubernetes léger (K3s). Il configure le contrôleur d'entrée (Nginx Ingress) et extrait le fichier de configuration **kubeconfig**, permettant une interaction programmatique avec le cluster depuis l'extérieur.

- **Manifestes Kubernetes :** Les développeurs formalisent l'état souhaité du système via des fichiers YAML définissant les déploiements, les services de communication (ClusterIP), les règles de routage (Ingress) et les mécanismes de persistance (PV/PVC) pour la base de données SQLite.

B. Zone 2 : Le Moteur CI/CD (GitHub & GitHub Actions)

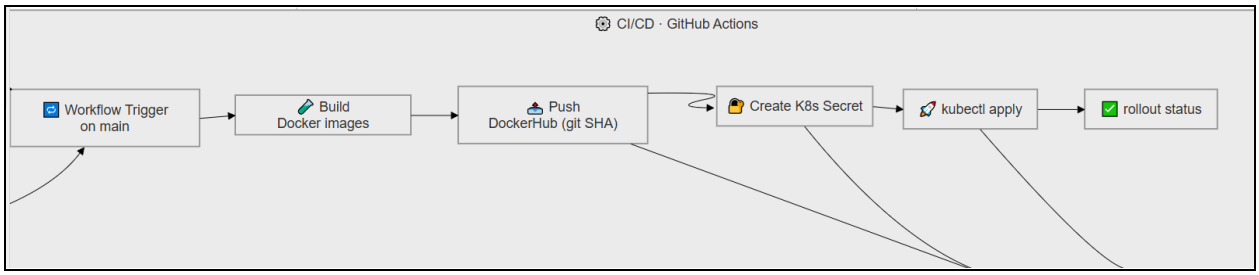
Cette zone agit comme un pont automatisé et sécurisé entre le code source et l'environnement d'exécution.



Une fois l'infrastructure prête, GitHub devient le centre de commandement :

- **Cycle de Développement et Collaboration :** Le projet utilise une stratégie de branches de fonctionnalités (**Feature Branches**). Toute modification de code fait l'objet d'une **Pull Request (PR)** et d'une revue de code avant d'être fusionnée (**Merge**) dans la branche principale.

- **Gestion des Secrets :** es données sensibles (identifiants DockerHub, certificats Kubeconfig, clés secrètes JWT) sont centralisées dans les **GitHub Secrets**. Cette approche prévient toute fuite de credentials au sein du code source.

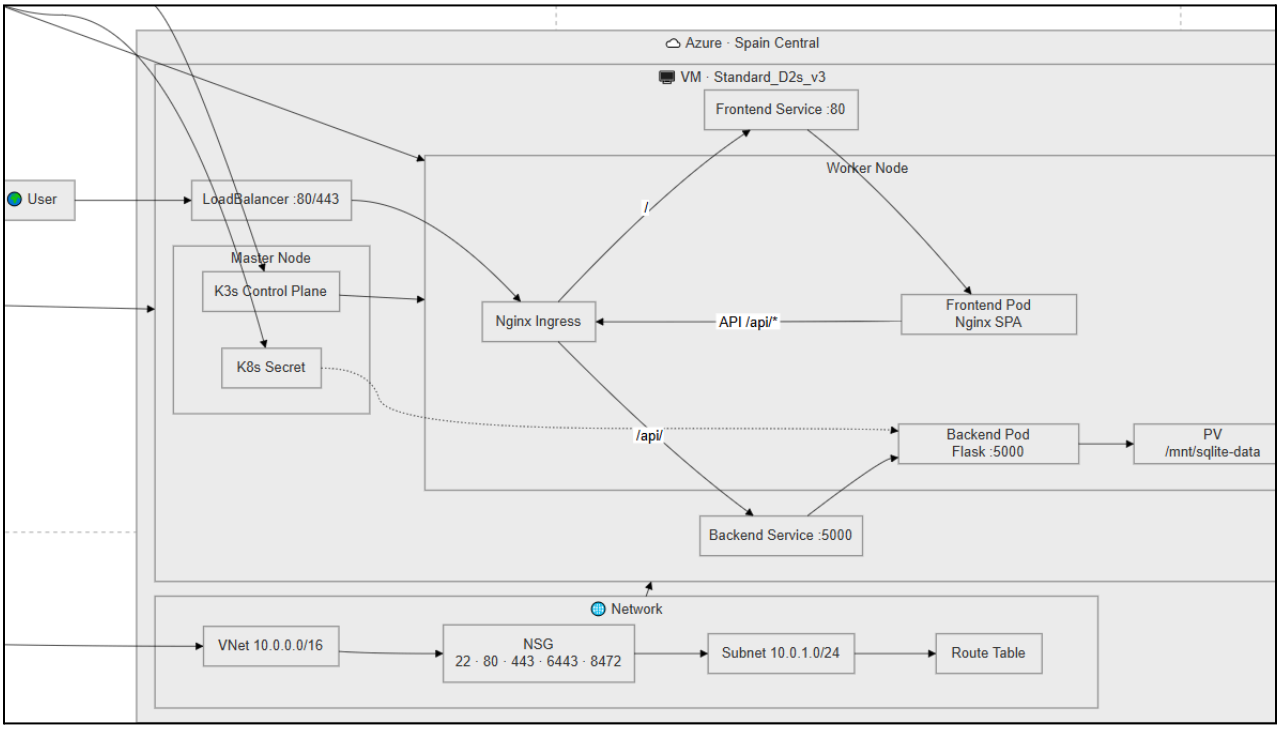


Pipeline de Livraison Continue : À chaque fusion sur la branche principale (main), un workflow automatisé exécute les étapes suivantes :

1. **Construction (Build) :** Génération des images Docker pour le Frontend (Nginx) et le Backend (Flask/Gunicorn).
2. **Versionnement :** Marquage des images avec le Git SHA pour garantir une traçabilité totale et faciliter les procédures de retour arrière (rollback).
3. **Distribution (Push) :** Publication des images sur le registre DockerHub.
4. **Injection de Secrets :** Création dynamique des secrets Kubernetes dans le cluster pour une consommation par les Pods au runtime.
5. **Déploiement et Validation :** Application des manifests et vérification de l'état de déploiement via **kubectl rollout status**, garantissant une mise à jour sans interruption de service.

C. Zone 3 : L'Environnement de Production (Azure Spain Central)

L'infrastructure finale est hébergée sur la région Azure Spain Central, exploitant la résilience et l'évolutivité de K3s.



1. Isolation et Sécurité Réseau

Le réseau virtuel est segmenté pour n'autoriser que les flux nécessaires : le port 22 pour l'administration (SSH), les ports 80/443 pour le trafic applicatif, et le port 6443 pour l'API Kubernetes. La communication inter-nœuds est assurée par le protocole VXLAN sur le port 8472.

2. Architecture du Cluster K3s

- **Nœud Master (Control Plane) :** Il assure la planification des Pods, la gestion de l'état du cluster et l'exposition de l'API Server. Le composant Traefik par défaut a été désactivé au profit d'une solution Nginx Ingress pour un contrôle plus granulaire du routage.
- **Nœud Worker (Data Plane) :** Il héberge la charge applicative.
    - **Nginx Ingress Controller :** Gère le routage de niveau 7 (HTTP) basé sur les chemins d'URL (/ pour le Frontend, /api/ pour le Backend).
    - **Pod Frontend :** Serveur Nginx servant l'application React en mode SPA (*Single Page Application*).
    - **Pod Backend :** Serveur Flask optimisé par Gunicorn, traitant les requêtes d'authentification et de prédiction.

- **Persistance des Données :** Un volume persistant (PersistentVolume) basé sur un hostPath est monté sur le nœud Worker. Ce mécanisme garantit que les données de la base SQLite sont conservées malgré les cycles de vie éphémères des conteneurs.

Conclusion de l'Architecture

Cette architecture répond aux exigences de l'ingénierie DevOps moderne par une automatisation intégrale, une séparation claire des environnements et une gestion intelligente de la disponibilité et de la persistance des données.

II. Conception Logicielle et Conteneurisation

2.1 Le Backend (Flask & Gunicorn)

Le backend constitue le cœur décisionnel du système. Il héberge le moteur de recommandation basé sur un modèle de Machine Learning préalablement entraîné.

```
from routes.auth import auth_bp
from routes.predict import predict_bp
from routes.student import student_bp
app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(student_bp, url_prefix='/api/student')
app.register_blueprint(predict_bp, url_prefix='/api/predict')
```

- **Logique du moteur :** L'API Flask expose des endpoints comme /api/predict qui reçoit les notes de l'étudiant, les transforme en vecteurs numériques et interroge le modèle pour retourner la spécialisation suggérée.

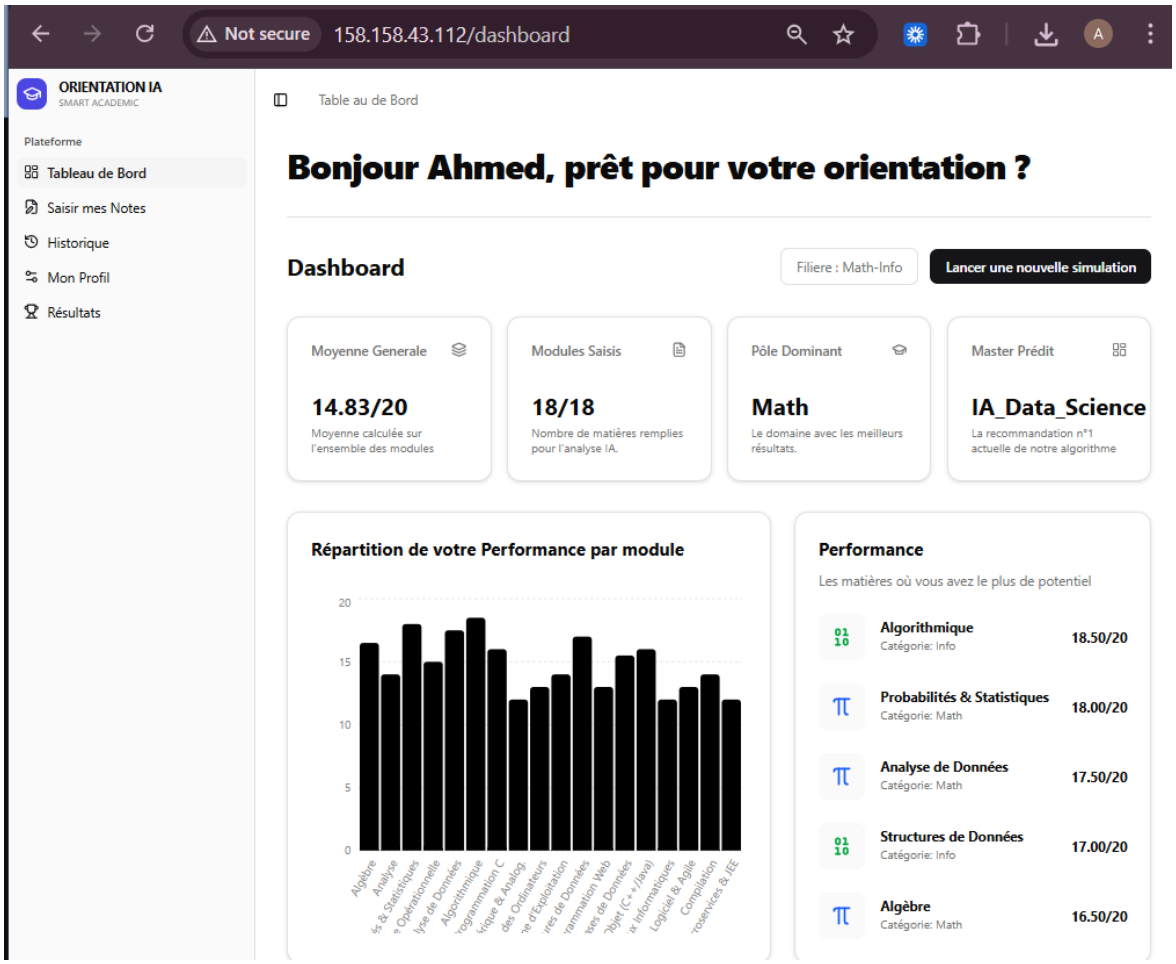
```
FROM python:3.11-slim

WORKDIR /app

# System dependencies for some ML libraries
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
# Add gunicorn to  requirements.txt or install it here
RUN pip install --no-cache-dir -r requirements.txt gunicorn
```

```
COPY . .

# Ensure the instance folder exists for SQLite
RUN mkdir -p instance

EXPOSE 5000

# Use Gunicorn: 2 workers
# 'app:app' means look in app.py for the 'app' variable.
# one wroker for 1GB RAM ?
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "app:app"]
```

- **Pourquoi Gunicorn ?** Le serveur natif de Flask (Werkzeug) est un serveur de développement qui ne gère qu'une requête à la fois. Nous utilisons **Gunicorn** (*Green Unicorn*), un serveur WSGI de production, pour permettre le parallélisme via des "workers". Cela garantit que l'application reste réactive même en cas de requêtes simultanées.
- **Analyse du Dockerfile Backend :**
  - **Image de base :** python:3.11-slim pour minimiser la taille de l'image et la surface d'attaque.
  - **Optimisation des couches :** L'installation des dépendances (requirements.txt) est isolée avant la copie du code source pour exploiter le cache Docker et accélérer les builds lors des mises à jour de code.

2.2 Le Frontend (React & Nginx)

L'interface utilisateur a été conçue pour offrir une expérience fluide (UX) et une saisie intuitive des données académiques.



- **Interface et Expérience :** Développée avec React et la bibliothèque de composants **shadcn/ui**, l'application assure une validation des données côté client avant l'envoi au backend.

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

- **Configuration nginx.conf :** Le frontend étant une *Single Page Application* (SPA), le routage est géré par le navigateur. Nous avons configuré Nginx avec la directive try_files $uri $uri/ /index.html;. Cela force Nginx à renvoyer index.html même si l'utilisateur rafraîchit une route spécifique (ex: /result), laissant React gérer l'affichage.

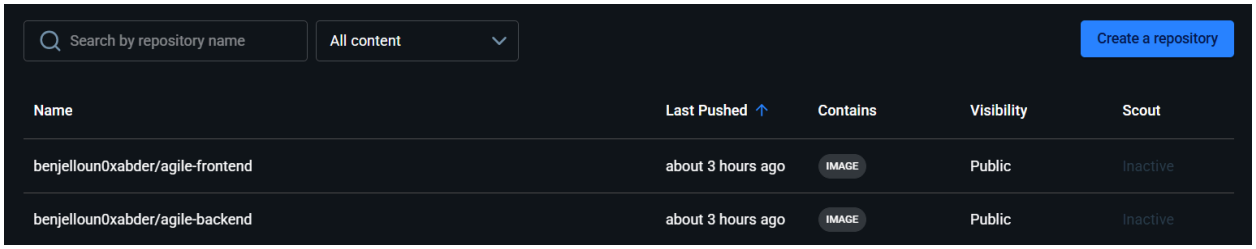
```
# STAGE 1: Build the React App
FROM node:22-alpine AS build-stage
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

```
# STAGE 2: Serve with Nginx
FROM nginx:stable-alpine
# Copy the built files from the first stage
COPY --from=build-stage /app/dist /usr/share/nginx/html
# Copy our portable config
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

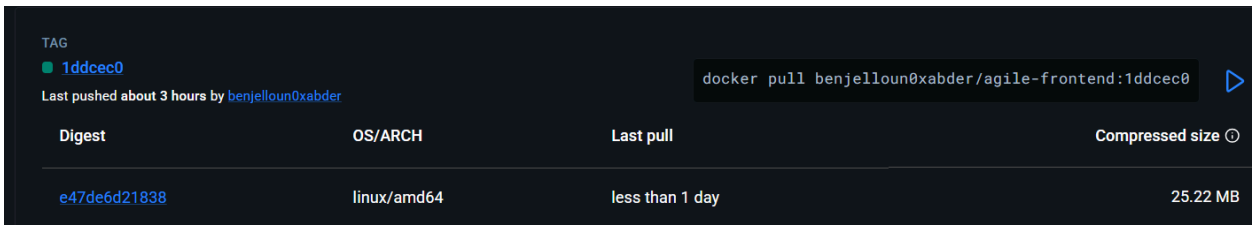
- **Multi-stage Build :** Le Dockerfile utilise deux étapes :
    1. **Stage Build :** Compilation du code source (Node.js) pour générer les fichiers statiques optimisés.
    2. **Stage Production :** Copie uniquement des fichiers compilés dans une image Nginx légère. Cela réduit la taille de l'image finale de 80% en éliminant les outils de développement.

2.3 Registre d'images : Docker Hub

Le registre Docker Hub agit comme le pivot central de la chaîne de distribution.



- **Découplage :** Il permet de séparer l'étape de construction (GitHub Actions) de l'étape d'exécution (Azure).



- **Immutabilité :** En poussant des images taguées avec le Git SHA, nous garantissons que le code testé en CI est exactement le même que celui déployé sur le cluster K3s.

III. Infrastructure as Code (IaC) avec Terraform

Le provisionnement de l'infrastructure est entièrement piloté par **Terraform**. Cette approche permet de définir l'état souhaité de notre architecture Azure dans des fichiers de configuration versionnés, garantissant une reproductibilité totale.

3.1 Provisionnement sur Azure (main.tf)

Le déploiement repose sur une architecture Cloud robuste segmentée dans la région **Spain Central**.

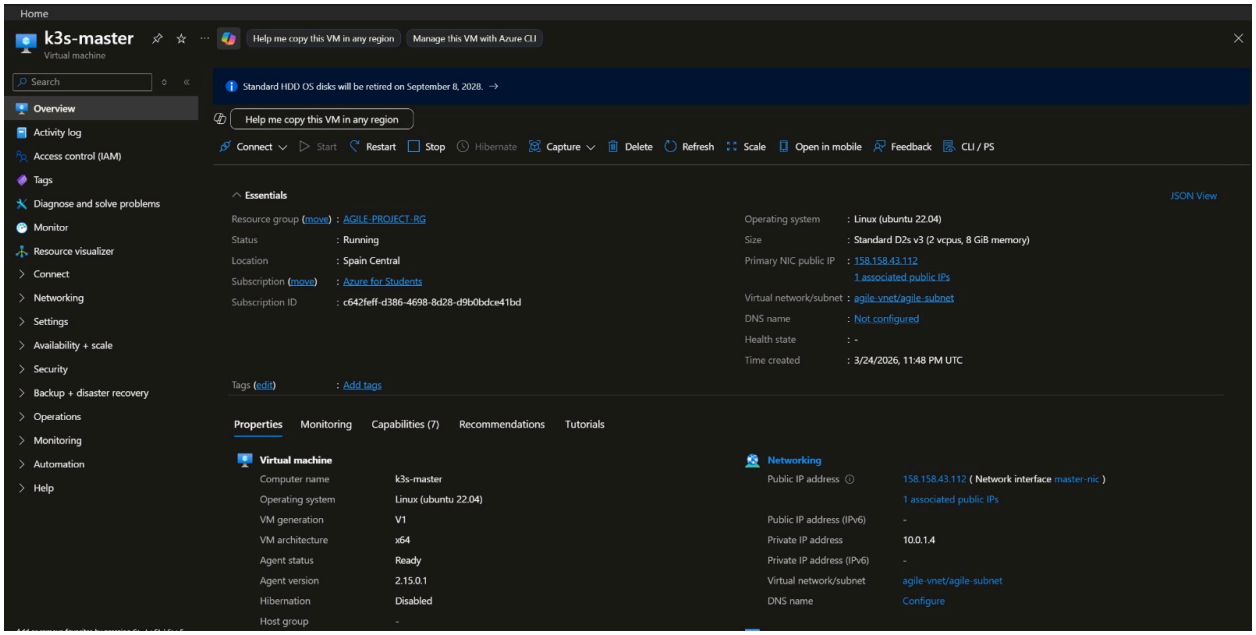
```
resource "azurerm_virtual_network" "main" {
  name                = "agile-vnet"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_subnet" "main" {
  name                 = "agile-subnet"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

- **Réseau et Segmentation :** Nous avons défini un réseau virtuel (azurerm_virtual_network) nommé agile-vnet avec un espace d'adressage 10.0.0.0/16. Le sous-réseau 10.0.1.0/24 accueille nos instances, isolant ainsi le cluster du reste du cloud.

```
resource "azurerm_linux_virtual_machine" "worker" {
  name                = "k3s-worker"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  size                = "Standard_D2s_v3"
  admin_username      = "ubuntu"
```

- **Choix des Instances :** Nous utilisons des machines de taille **Standard_D2s_v3** (Ubuntu 22.04 LTS). Ce choix est stratégique : avec 2 vCPUs et 8 Go de RAM, elles offrent les ressources nécessaires pour faire tourner simultanément le moteur de recommandation IA et les services Kubernetes sans saturation de la mémoire.

- **Cycle de vie Terraform :** Le déploiement suit un flux rigoureux :
  1. terraform plan : Pour valider les changements et visualiser l'impact sur Azure.
  2. terraform apply : Pour exécuter le provisionnement réel des 13 ressources définies.



3.2 Sécurité Réseau (NSG)

La sécurité est orchestrée via un **Network Security Group (NSG)** nommé agile-nsg. Nous appliquons le principe du moindre privilège en n'ouvrant que les flux critiques :

Règle	Port	Protocole	Description
SSH	22	TCP	Administration à distance et déploiement Ansible.
HTTP/S	80 / 443	TCP	Accès public à l'interface React et à l'API.
K3sAPI	6443	TCP	Accès distant au serveur d'API Kubernetes.
FlannelVXLAN	8472	UDP	Communication réseau interne entre les nœuds du cluster.
Kubelet	10250	TCP	Métriques et logs internes du cluster.

3.3 Intégration Ansible : Inventaire Automatisé

L'un des points les plus avancés de notre configuration est l'utilisation du provider local_file pour automatiser la liaison entre l'infrastructure et la configuration

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}
```

```
resource "local_file" "ansible_inventory" {
  content = <<EOT
[master]
master_node ansible_host=${azurerm_public_ip.master.ip_address}

[worker]
worker_node ansible_host=${azurerm_public_ip.worker.ip_address}

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/my-key
ansible_python_interpreter=/usr/bin/python3
EOT
  filename = "../ansible/inventory.ini"
}
```

- **Génération de l'inventaire :** Dès que les ressources sont provisionnées, Terraform récupère dynamiquement les adresses IP publiques (azurerm_public_ip) du Master et du Worker.
- **Injection Dynamique :** Ces adresses sont injectées dans un template pour générer automatiquement le fichier ../ansible/inventory.ini.
- **Résultat :** Cela permet de lancer immédiatement le déploiement Ansible sans avoir à éditer manuellement les fichiers de configuration, éliminant ainsi les erreurs de saisie d'IP.

IV. Configuration Automatisée avec Ansible

Une fois l'infrastructure provisionnée, nous utilisons **Ansible** pour transformer les instances nues en un cluster Kubernetes cohérent. L'exécution est pilotée par un fichier ansible.cfg optimisé, désactivant le host_key_checking pour fluidifier le déploiement automatisé sur Azure.

4.1 Préparation du Runtime (Rôle common)

Le rôle common garantit que chaque nœud dispose des outils nécessaires avant l'installation de l'orchestrateur.

```
- name: Update apt cache
  apt:
    update_cache: yes
    cache_valid_time: 3600

- name: Install required system packages
  apt:
    name:
      - apt-transport-https
      - ca-certificates
      - curl
      - gnupg
      - lsb-release
    state: present
```

- **Mise à jour et Dépendances :** Le playbook assure la mise à jour du cache APT et l'installation des paquets de base (apt-transport-https, ca-certificates, curl, gnupg).

```yaml
- name: Add Docker GPG key
  apt_key:
    url: https://download.docker.com/linux/ubuntu/gpg
    state: present

- name: Add Docker repository
  apt_repository:
    repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }} stable"
    state: present

- name: Install Docker
  apt:
    name:
      - docker-ce
      - docker-ce-cli
      - containerd.io
    state: present
    update_cache: yes
```

- **Moteur de Conteneurisation :** Nous automatisons l'ajout de la clé GPG et du dépôt officiel de **Docker**. Bien que K3s intègre son propre runtime, Docker est installé sur tous les nœuds pour offrir une flexibilité de gestion des images et une compatibilité maximale avec les utilitaires système.

```yaml
- name: Add ubuntu user to docker group
  user:
    name: ubuntu
    groups: docker
    append: yes
```

- **Privilèges :** L'utilisateur ubuntu est automatiquement ajouté au groupe docker pour permettre une administration sans élévation de privilèges systématique.

4.2 Déploiement et Initialisation du Cluster (Rôle k3s_master)

L'installation du Master est l'étape critique où le plan de contrôle est défini.

```
- name: Install K3s master
  shell: |
    curl -sfL https://get.k3s.io | sh -s - server \
      --disable traefik \
      --write-kubeconfig-mode 644 \
      --tls-san {{ ansible_host }}
  args:
    creates: /usr/local/bin/k3s
```

- **Configuration du Serveur :** Le script d'installation de K3s est exécuté avec des paramètres spécifiques :
  - --disable traefik : Nous désactivons le contrôleur par défaut pour déployer manuellement **Nginx Ingress Controller**, offrant un meilleur contrôle du routage.
  - --tls-san {{ ansible_host }} : Crucial pour la sécurité, ce paramètre autorise l'accès distant au serveur d'API via l'IP publique.

```
- name: Wait for K3s to be ready
  wait_for:
    path: /var/lib/rancher/k3s/server/node-token
    timeout: 60

- name: Get K3s node token (Worker needs this to join)
  slurp:
    src: /var/lib/rancher/k3s/server/node-token
  register: k3s_token

- name: Save token as a fact for the worker role to use
  set_fact:
    k3s_node_token: "{{ k3s_token.content | b64decode | trim }}"
```

```
- name: Get Master private IP
  set_fact:
    master_private_ip: "{{ ansible_default_ipv4.address }}"

- name: Share token and IP with the worker group
  add_host:
    name: "k3s_token_holder"
    k3s_node_token: "{{ k3s_node_token }}"
    master_private_ip: "{{ master_private_ip }}"
```

- **Transmission du Jeton (Token) :** Le rôle utilise le module slurp pour lire le fichier /var/lib/rancher/k3s/server/node-token. Ce jeton est enregistré comme une "fact" Ansible et partagé dynamiquement avec le groupe de workers via add_host.

```
- name: Install Nginx Ingress Controller
  shell: kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.0/deploy/stat
  environment:
    KUBECONFIG: /etc/rancher/k3s/k3s.yaml
```

- **Routage de Niveau 7 :** Le playbook déploie le contrôleur Ingress Nginx officiellement via kubectl apply, préparant ainsi le cluster à recevoir le trafic HTTP/HTTPS.

4.3 Jonction du Nœud de Travail (Rôle k3s_worker)

Le rôle worker assure l'extension du cluster par une jonction sécurisée.

```
- name: Install K3s worker and join cluster
  shell: |
    curl -sfL https://get.k3s.io | K3S_URL=https://{{ hostvars['k3s_token_holder']['master_private_ip'] }}:6443 \
      K3S_TOKEN={{ hostvars['k3s_token_holder']['k3s_node_token'] }} sh -
  args:
    creates: /usr/local/bin/k3s
```

- **Connexion au Master :** Le worker utilise l'IP privée du Master et le jeton récupéré précédemment pour s'enregistrer auprès du cluster. L'utilisation de l'IP privée garantit que le trafic interne reste sécurisé au sein du VNet Azure.

```
- name: Wait for K3s agent to start
  systemd:
    name: k3s-agent
    state: started
    enabled: yes
```

- **Persistance du Service :** Le playbook active et démarre l'agent k3s-agent via systemd, assurant que le worker se reconnecte automatiquement au Master en cas de redémarrage de la VM.

4.4 Extraction et Préparation du Kubeconfig

Pour permettre le pilotage du cluster depuis l'extérieur (machine locale et GitHub Actions), le rôle Master termine par une étape d'extraction :

- **Récupération (Fetch) :** Le fichier /etc/rancher/k3s/k3s.yaml est rapatrié localement sous le nom k3s-kubeconfig.yaml.
- **Portabilité :** Ce fichier sera ultérieurement modifié (remplacement de l'IP locale par l'IP publique du Master) pour être injecté dans les **GitHub Secrets**, fermant ainsi la boucle de l'automatisation CI/CD.

V. Orchestration et Résilience du Cluster (K3s)

L'orchestration Kubernetes constitue le pivot opérationnel du projet. Elle transforme des images statiques en une application dynamique, capable de s'auto-réparer et de gérer intelligemment le trafic utilisateur tout en respectant les contraintes de ressources de nos instances Azure.

5.1 Abstraction des Workloads et Contrôle d'État

Le déploiement des microservices ne se fait pas de manière isolée mais via des objets Deployment qui garantissent l'immuabilité et la résilience.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 1
```

- **Gestion du Desired State :** Chaque unité de calcul (Pod) est supervisée par un *ReplicaSet*. En cas de défaillance du processus Flask ou Nginx, le plan de contrôle Kubernetes détecte l'écart avec l'état souhaité et réinstancie automatiquement un nouveau Pod.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: backend-secrets
type: Opaque
stringData:
  JWT_SECRET_KEY: "REPLACED_BY_GHA"
  DATABASE_URL: "REPLACED_BY_GHA"
```

```yaml
envFrom:
  # All keys from the Secret are injected as environment
  - secretRef:
      name: backend-secrets
```

- **Injection Dynamique de Secrets :** Pour le Backend, nous utilisons l'objet Secret. Les clés sensibles (JWT, Database URL) sont montées en variables d'environnement via la directive envFrom. Cela permet de découpler la configuration sécurisée du code binaire contenu dans l'image Docker.

```yaml
containers:
  - name: backend
    image: YOUR_DOCKERHUB_USERNAME/agile-backend:latest
    # GHA will replace 'latest' with the git SHA tag e.g. :a1
```

- **Stratégie de Mise à Jour (Rolling Update) :** Le pipeline CI/CD déclenche une mise à jour progressive. Kubernetes déploie la nouvelle version de l'image (marquée par le Git SHA) et ne résilie les anciens Pods qu'une fois les nouveaux déclarés "Ready", assurant une disponibilité continue.

5.2 Topologie Réseau et Exposition des Services

L'architecture réseau est conçue pour isoler les flux et optimiser le routage des requêtes.

A. Communication Inter-services (ClusterIP)

Le Frontend et le Backend sont encapsulés derrière des **Services de type ClusterIP**.

```yaml
# Frontend Service - ClusterIP, Ingress routes to it
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```


```yaml
# Frontend Service - ClusterIP, Ingress routes to it
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```


- **Mécanique :** Le service fournit une IP virtuelle stable et un nom DNS interne au cluster.
- **Sécurité :** Ces composants ne possèdent pas d'IP publique directe. Le Backend n'est accessible que par le Frontend ou le contrôleur d'entrée, réduisant drastiquement la surface d'attaque du serveur d'API (port 5000).

B. Routage de Couche 7 (Ingress NGINX)

L'accès externe est centralisé par un contrôleur **Ingress NGINX** qui opère un routage basé sur les chemins d'URL (*Path-based routing*) :

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: agile-ingress
  annotations:
    # This tells K8s which Ingress Controller to use
    kubernetes.io/ingress.class: "nginx"
```

```yaml
# All other requests go to the React frontend
- path: /
  pathType: Prefix
  backend:
    service:
      name: frontend-service
      port:
        number: 80
```

- **Règle Frontend (/) :** Redirige le trafic vers le service React. Le serveur Nginx du conteneur gère alors le routage applicatif (SPA).

```yaml
paths:
  # All /api/* requests go to the Flask backend
  - path: /api/
    pathType: Prefix
    backend:
      service:
        name: backend-service
        port:
          number: 5000
```

- **Règle API (/api/) :** Toutes les requêtes préfixées par /api sont routées vers l'amont (*Upstream*) Flask. Cette configuration permet de mutualiser le port 80/443 pour l'ensemble des microservices tout en gérant les en-têtes HTTP de manière centralisée.

5.3 Persistance des Données (PV/PVC)

Étant donné la nature éphémère des Pods, la persistance de la base de données SQLite nécessite une abstraction du stockage physique.

```yaml
# PersistentVolume - maps to a real folder on the EC2 w
apiVersion: v1
kind: PersistentVolume
metadata:
  name: sqlite-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  # hostPath means it uses the EC2 node's local disk
  hostPath:
    path: /mnt/sqlite-data
```

- **Le PersistentVolume (PV) :** Définit une ressource de stockage statique de 1 Go liée au disque local du nœud Worker via un hostPath (/mnt/sqlite-data).

```yaml
---

# PersistentVolumeClaim - the backend pod requests storage
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sqlite-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

- **Le PersistentVolumeClaim (PVC) :** Le Pod Backend émet une requête de réservation de stockage. Kubernetes effectue un *Binding* automatique entre le PVC et le PV disponible.

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ kubectl get pvc
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
sqlite-pvc    Bound    pvc-3b5962fe-eb8f-422c-9e6a-9436f2aa3d10   1Gi        RWO            local-path     <unset>                 169m
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$
```

- La commande kubectl get pvc montre le statut Bound et la capacité utilisée.

```yaml
          volumeMounts:
            - name: sqlite-storage
              mountPath: /app/instance
      volumes:
        - name: sqlite-storage
          persistentVolumeClaim:
            claimName: sqlite-pvc
```

- **Volume Mounting :** Le volume est monté dans le conteneur Backend sur /app/instance. Ce mécanisme garantit que les transactions SQL sont écrites physiquement sur le nœud Azure, permettant aux données de survivre aux redémarrages ou aux mises à jour du Pod.

VI. Automatisation CI/CD avec GitHub Actions

Le pipeline d'automatisation est conçu pour éliminer les erreurs humaines et réduire le délai de mise en production (*Time-to-Market*). En s'appuyant sur **GitHub Actions**, nous avons implémenté un workflow capable de gérer l'intégralité du cycle de vie des conteneurs.

6.1 Architecture du Workflow de Déploiement

Le fichier deploy.yml orchestre une suite logique de tâches déclenchées à chaque fusion sur la branche **main**.

```yaml
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set image tag
        run: echo "IMAGE_TAG=${GITHUB_SHA::7}" >> $GITHUB_ENV
```

- **Récupération et Environnement :** Le workflow initialise l'agent ubuntu-latest, récupère le code source et génère une variable d'environnement IMAGE_TAG basée sur les 7 premiers caractères du **GITHUB_SHA**.

```yaml
# ── STEP 1: Build and push images to DockerHub ──────────────────────

- name: Login to DockerHub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}

- name: Build and push backend image
  run: |
    docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/agile-backend:${{ env.IMAGE_TAG }} ./agile-backend
    docker push ${{ secrets.DOCKERHUB_USERNAME }}/agile-backend:${{ env.IMAGE_TAG }}

- name: Build and push frontend image
  run: |
    docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/agile-frontend:${{ env.IMAGE_TAG }} ./agile-frontend
    docker push ${{ secrets.DOCKERHUB_USERNAME }}/agile-frontend:${{ env.IMAGE_TAG }}
```

- **Phase de Construction (Build & Push) :** Le pipeline s'authentifie sur **Docker Hub** via des jetons sécurisés.
    - Il construit les images pour le Frontend et le Backend. Contrairement à un build local, cette étape garantit que les images sont produites dans un environnement propre et standardisé.

```yaml
- name: Set up kubeconfig
  run: |
    mkdir -p ~/.kube
    echo "${{ secrets.KUBECONFIG }}" > ~/.kube/config
    # KUBECONFIG secret = contents of k3s-kubeconfig.yaml from Ansible
    # You must replace the server IP: sed -i 's/127.0.0.1/MASTER_PUBLIC_IP/' ~/.kube/config
    sed -i 's/127.0.0.1/${{ secrets.MASTER_PUBLIC_IP }}/' ~/.kube/config
```

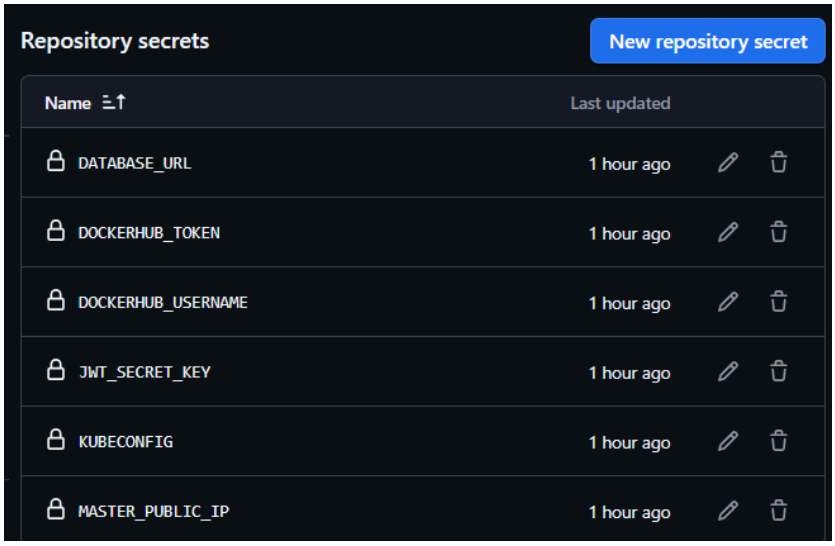
- **Phase de Déploiement (Continuous Deployment) :** Le workflow configure dynamiquement le fichier kubeconfig en remplaçant l'adresse 127.0.0.1 par l'adresse IP publique du Master (stockée en secret).

```yaml
- name: Update image tags in manifests and apply
  run: |
    # Replace YOUR_DOCKERHUB_USERNAME/agile-backend:latest with actual tag
    sed -i 's|YOUR_DOCKERHUB_USERNAME/agile-backend:latest|${{ secrets.DOCKERHUB_USE
    sed -i 's|YOUR_DOCKERHUB_USERNAME/agile-frontend:latest|${{ secrets.DOCKERHUB_US

    kubectl apply -f kubernetes/persistent-volume.yaml
    kubectl apply -f kubernetes/backend-deployment.yaml
    kubectl apply -f kubernetes/frontend-deployment.yaml
    kubectl apply -f kubernetes/services.yaml
    kubectl apply -f kubernetes/ingress.yaml
```

    - Il procède ensuite à la mise à jour des ressources Kubernetes par une application séquentielle des manifestes.

6.2 Gestion et Sécurisation des Secrets

Pour respecter les normes de sécurité **DevSecOps**, le pipeline n'utilise aucune donnée sensible en clair.



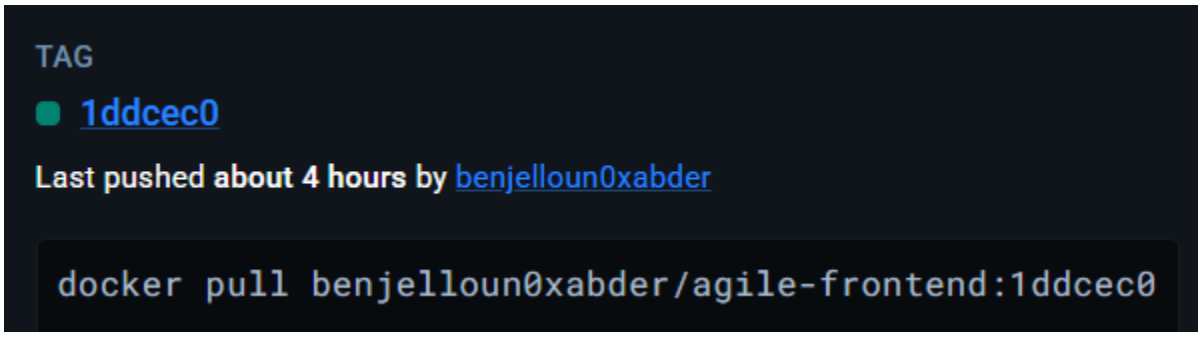
- **Coffre-fort GitHub :** Les identifiants (DOCKERHUB_TOKEN), les fichiers de configuration du cluster (KUBECONFIG) et les variables applicatives (JWT_SECRET_KEY) sont stockés dans les **GitHub Secrets**.

```
- name: Create backend secret from GitHub Secrets
  run: |
    kubectl create secret generic backend-secrets \
      --from-literal=JWT_SECRET_KEY=${{ secrets.JWT_SECRET_KEY }} \
      --from-literal=DATABASE_URL=${{ secrets.DATABASE_URL }} \
      --dry-run=client -o yaml | kubectl apply -f -
```

- **Injection Dynamique de Secrets Kubernetes :** Plutôt que de stocker un fichier YAML de secret statique sur le dépôt, le pipeline génère le secret Kubernetes backend-secrets à la volée via la commande kubectl create secret --dry-run=client -o yaml. Cela permet de transmettre les variables d'environnement au Pod Backend de manière totalement sécurisée au moment du déploiement.

6.3 Stratégie de Tagging et Immutabilité

L'utilisation du tag latest est proscrite en environnement de production car elle ne permet pas d'identifier avec certitude la version du code exécutée.



- **Utilisation du GITHUB_SHA :** Nous utilisons l'empreinte unique du commit Git pour taguer nos images Docker.

```
- name: Wait for rollout
  run: |
    kubectl rollout status deployment/backend
    kubectl rollout status deployment/frontend
```

- **Immutabilité :** Cette approche garantit que chaque version déployée est immuable. Si un bogue est détecté, nous pouvons identifier précisément le commit responsable et effectuer un retour arrière (*Rollback*) vers une version précédente en pointant simplement sur l'ancien tag SHA.

```
- name: Update image tags in manifests and apply
  run: |
    # Replace YOUR_DOCKERHUB_USERNAME/agile-backend:latest with actual tag
    sed -i 's|YOUR_DOCKERHUB_USERNAME/agile-backend:latest|${{ secrets.DOCKERHUB_USERNAME }}/agile-backend:${{ env.IMAGE_TAG }}|g' kubernetes/backend-deployme
    sed -i 's|YOUR_DOCKERHUB_USERNAME/agile-frontend:latest|${{ secrets.DOCKERHUB_USERNAME }}/agile-frontend:${{ env.IMAGE_TAG }}|g' kubernetes/frontend-deplo
```

- **Mise à jour des Manifestes :** Le pipeline utilise la commande sed pour modifier "à la volée" les fichiers de déploiement avant de les appliquer au cluster. Cela assure que Kubernetes télécharge toujours la version exacte produite durant la phase de build actuelle.

VIII. Guide de Reproduction (Documentation)

Ce guide détaille la procédure permettant de répliquer l'environnement complet, de la création des ressources Azure au déploiement automatisé via GitHub Actions.

8.1 Pré-requis et Préparation des Identifiants

Avant de commencer, assurez-vous d'avoir installé les outils suivants sur votre machine locale (Linux/WSL2 recommandé) : **Terraform**, **Ansible**, **kubectl** et **Azure CLI**.

-Installation Terrarform

```
benjo@DESKTOP-4GCHLHE:~$ sudo apt update && sudo apt install -y terraform
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
Get:5 https://apt.releases.hashicorp.com jammy InRelease [12.9 kB]
Hit:6 https://packages.redis.io/deb jammy InRelease
Get:7 https://apt.releases.hashicorp.com jammy/main amd64 Packages [225 kB]
Fetched 238 kB in 4s (65.9 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
116 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  terraform
0 upgraded, 1 newly installed, 0 to remove and 116 not upgraded.
Need to get 30.6 MB of archives.
After this operation, 101 MB of additional disk space will be used.
Get:1 https://apt.releases.hashicorp.com jammy/main amd64 terraform amd64 1.14.7-1 [30.6 MB]
Fetched 30.6 MB in 6s (5507 kB/s)
Selecting previously unselected package terraform.
(Reading database ... 37406 files and directories currently installed.)
Preparing to unpack .../terraform_1.14.7-1_amd64.deb ...
Unpacking terraform (1.14.7-1) ...
Setting up terraform (1.14.7-1) ...
```

-Installing ansible

```
Processing triggers for man-db (2.10.2-1) ...
benjo@DESKTOP-4GCHLHE:~$ sudo apt install -y ansible
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-argcomplete python3-jmespath python3-kerberos python3-libcloud python3-l
  python3-requests-kerberos python3-requests-ntlm python3-selinux python3-simplejs
Suggested packages:
  cowsay sshpass python-lockfile-doc
The following NEW packages will be installed:
  python3-argcomplete python3-jmespath python3-kerberos python3-libcloud python3-l
  python3-requests-kerberos python3-requests-ntlm python3-selinux python3-simplejs
The following packages will be upgraded:
```

```
benjo@DESKTOP-4GCHLHE:~$ ansible --version
ansible 2.10.8
  config file = None
  configured module search path = ['/home/benjo/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.10.12 (main, Jan 26 2026, 14:55:28) [GCC 11.4.0]
benjo@DESKTOP-4GCHLHE:~$
```

-Installation kubectl :

```
benjo@DESKTOP-4GCHLHE:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linu
x/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 55.8M  100 55.8M    0     0  6014k      0  0:00:09  0:00:09 --:--:-- 6310k
benjo@DESKTOP-4GCHLHE:~$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
[sudo] password for benjo:
benjo@DESKTOP-4GCHLHE:~$ kubectl version --client
Client Version: v1.35.3
Kustomize Version: v5.7.1
```

Obtention des identifiants Azure

Pour permettre à Terraform de piloter votre compte Azure, vous devez générer un "Service Principal". Exécutez les commandes suivantes dans votre terminal :

1. **Récupérer l'ID de souscription :** az account show --query id --output tsv
2. **Créer les accès (remplacer SUBSCRIPTION_ID) :** az ad sp create-for-rbac --name "terraform-sp" --role Contributor --scopes /subscriptions/SUBSCRIPTION_ID

Note : Conservez précieusement les valeurs appId, password et tenant qui s'affichent

Étape 1 : Génération des clés de sécurité

Créez la paire de clés SSH qui permettra à Ansible de configurer les machines : ssh-keygen -t rsa -b 4096 -f ~/.ssh/my-key

```
benjo@DESKTOP-4GCHLHE:~$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/my-key
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/benjo/.ssh/my-key
Your public key has been saved in /home/benjo/.ssh/my-key.pub
The key fingerprint is:
SHA256:kFsERiJbJfECvklrVDBi2DU0q2qHUaKJLrOTrqGegsg benjo@DESKTOP-4GCHLHE
The key's randomart image is:
+---[RSA 4096]----+
|oo=+@+=..        |
|oo.B O o         |
| .=.o + .        |
|o+o= . +         |
|+.*   . S        |
|.oo              |
|O= .             |
|XEo              |
|O=               |
+----[SHA256]-----+
benjo@DESKTOP-4GCHLHE:~$
```

Étape 2 : Configuration des variables Terraform

Dans le dossier terraform/, créez un fichier terraform.tfvars et renseignez les identifiants obtenus précédemment :

```
subscription_id = "votre_id"
client_id       = "votre_appId"
client_secret   = "votre_password"
tenant_id       = "votre_tenant"
```

Étape 3 : Déploiement de l'Infrastructure

Initialisez et appliquez la configuration :

>terraform init

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Finding hashicorp/local versions matching "~> 2.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

>terraform plan

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/terraform$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_linux_virtual_machine.master will be created
  + resource "azurerm_linux_virtual_machine" "master" {
      + admin_username                                       = "ubuntu"
      + allow_extension_operations                           = true
      + bypass_platform_safety_checks_on_user_schedule_enabled = false
      + computer_name                                        = (known after apply)
      + disable_password_authentication                      = true
      + disk_controller_type                                 = (known after apply)
      + extensions_time_budget                               = "PT1H30M"
      + id                                                   = (known after apply)
      + location                                             = "westeurope"
      + max_bid_price                                        = -1
      + name                                                 = "k3s-master"
      + network_interface_ids                                = (known after apply)
      + patch_assessment_mode                                = "ImageDefault"
      + patch_mode                                           = "ImageDefault"
      + platform_fault_domain                                = -1
      + priority                                             = "Regular"
      + private_ip_address                                   = (known after apply)
      + private_ip_addresses                                 = (known after apply)
      + public_ip_addresses                                  = (known after apply)
      + resource_group_name                                  = "agile-rg"
      + size                                                 = "Standard_B1ms"
      + virtual_machine_id                                   = (known after apply)
      + vm_agent_platform_updates_enabled                    = false

      + admin_ssh_key {
          + public_key = <<-EOT
                ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQDbTm7Y9GSjp40KSaEsc/pcy0MrKoeAPmTXmUBbEXHeQx9ed4ZWrS48gsAtP2v3squv
GNqsKxaO5aIo9bAarscZIa3iNi28s6IZ3IRT4Fxd0gjU9A25SuBLc8A7v25x7LlNDCuDvvhHVuqHu9Mt3ojVw7aKfLHjK8tEUBAFp1VgxOojBkkGSiPr4OhH
CBrpXobHYyyQXGy5yim8Yxnn/gsQzGn1SwKAa5DoHZL6ZPkjl/knq+Oq/201zvGJV50rg1U6ppwoOa55528PBTxMWN2y8EPLZnVBszJ6Tt61xDihAJmUA8wx
BuWY5Hdfuf69oc1g+k4JnFcn0aVKHUVU5FXc2sLwVGsJq7LnMFw2JJLpYZpXvRYA6ONiFA4ceGHy+HsyKDEWJAIfANUQeNRKyK5rSZEwu6C750076n4rQAsW
NgoSLg8Ov7i3AJb3CW1v2WSSwhhYuKP3THn2uvCBzzxLokYjtDKtNBOK/2oUBrMXpKG7OBq+mR+ilfXdmQApPx3ziMLmo+fqtggB13uTWacgQ4VoSI2Q85Da
50Uj4KL5tok8ZOeScOiLuAEu5L4Y37tFPaMQVdebNrbIMlC16x+QCmei8vQqwlOFdvy9kfq3opwvur2zXzHl0izRzcGzR6lj6YPqoqbKEKuJ3cgxg0FTxzNP
1mNeQ7caCDDR1o2Blw== benjo@DESKTOP-4GCHLHE
            EOT
          + username   = "ubuntu"
        }

      + os_disk {
          + caching                   = "ReadWrite"
          + disk_size_gb              = (known after apply)
          + name                      = (known after apply)
          + storage_account_type      = "Standard_LRS"
          + write_accelerator_enabled = false
        }

      + source_image_reference {
          + offer     = "0001-com-ubuntu-server-jammy"
          + publisher = "Canonical"
```

```
Plan: 13 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + master_private_ip = (known after apply)
  + master_public_ip  = (known after apply)
  + worker_public_ip  = (known after apply)
```

>terraform apply

```
ansible_ssh_private_key_file=~/.ssh/my-key.pem
ansible_python_interpreter=/usr/bin/python3benjo@DESKTOP-4GCHLHE:~/edu-path-ai/terraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

```
azurerm_network_interface.worker: Creating...
local_file.ansible_inventory: Creating...
local_file.ansible_inventory: Creation complete after 0s [id=3860cae1ba78cd0c69ed74731a30b9bd7e9bfd0a]
azurerm_network_interface.worker: Creation complete after 3s [id=/subscriptions/c642feff-d386-4698-8d28-d9b0bdce41bd/resourceGroups/agile-pr
oject-rg/providers/Microsoft.Network/networkInterfaces/worker-nic]
azurerm_network_interface_security_group_association.worker: Creating...
azurerm_linux_virtual_machine.worker: Creating...
azurerm_network_interface_security_group_association.worker: Creation complete after 2s [id=/subscriptions/c642feff-d386-4698-8d28-d9b0bdce4
1bd/resourceGroups/agile-project-rg/providers/Microsoft.Network/networkInterfaces/worker-nic|/subscriptions/c642feff-d386-4698-8d28-d9b0bdce
41bd/resourceGroups/agile-project-rg/providers/Microsoft.Network/networkSecurityGroups/agile-nsg]
azurerm_linux_virtual_machine.worker: Still creating... [00m10s elapsed]
azurerm_linux_virtual_machine.worker: Still creating... [00m20s elapsed]
azurerm_linux_virtual_machine.worker: Creation complete after 27s [id=/subscriptions/c642feff-d386-4698-8d28-d9b0bdce41bd/resourceGroups/agi
le-project-rg/providers/Microsoft.Compute/virtualMachines/k3s-worker]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

master_private_ip = "10.0.1.4"
master_public_ip = "158.158.43.112"
worker_public_ip = "158.158.49.16"
```

À la fin, notez les adresses IP publiques (master_public_ip) affichées en sortie (ou generation automatique du fichier par main.tf du terraform).

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/terraform$ cat ../ansible/inventory.ini
[master]
master_node ansible_host=158.158.43.112

[worker]
worker_node ansible_host=158.158.49.16

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/my-key
ansible_python_interpreter=/usr/bin/python3
```

Étape 4 : Configuration Logicielle avec Ansible

Basculez dans le dossier ansible/. Terraform a déjà généré votre fichier inventory.ini.

1. **Tester la connexion :** ansible all -m ping (doit répondre "pong").

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ ansible all -m ping
worker_node | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
master_node | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

-autre méthode pour tester le ping par ssh vers le master_node et le worker_node:

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ ssh -i ~/.ssh/my-key ubuntu@158.158.43.112
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Mar 25 00:00:38 UTC 2026

  System load:  0.12              Processes:             122
  Usage of /:   5.7% of 28.89GB   Users logged in:       0
  Memory usage: 4%                IPv4 address for eth0: 10.0.1.4
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

```
Last login: Tue Mar 24 23:59:29 2026 from 196.117.60.212
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@k3s-master:~$
```

2. **Lancer l'installation :** ansible-playbook playbook.yml Cette étape installe Docker, K3s et configure le cluster en environ 10 minutes.

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ ansible-playbook -i inventory.ini playbook.yml

PLAY [all] *********************************************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [master_node]
ok: [worker_node]

TASK [common : Update apt cache] ***********************************************************************
changed: [master_node]
changed: [worker_node]

TASK [common : Install required system packages] *******************************************************
changed: [master_node]
```

```
TASK [common : Install required system packages] *******************************************************
changed: [master_node]
changed: [worker_node]

TASK [common : Add Docker GPG key] *********************************************************************
changed: [worker_node]
changed: [master_node]

TASK [common : Add Docker repository] ******************************************************************
changed: [worker_node]
changed: [master_node]

TASK [common : Install Docker] *************************************************************************
changed: [master_node]
changed: [worker_node]

TASK [common : Start and enable Docker] ****************************************************************
ok: [worker_node]
ok: [master_node]

TASK [common : Add ubuntu user to docker group] ********************************************************
changed: [master_node]
changed: [worker_node]

PLAY [master] ******************************************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [master_node]
```

```
TASK [Gathering Facts] ***********************************************************************************
ok: [master_node]

TASK [k3s_master : Install K3s master] *******************************************************************
[WARNING]: Consider using the get_url or uri module rather than running 'curl'.  If you need to use command because get_url
insufficient you can add 'warn: false' to this command task or set 'command_warnings=False' in ansible.cfg to get rid of thi
changed: [master_node]

TASK [k3s_master : Wait for K3s to be ready] *************************************************************
ok: [master_node]

TASK [k3s_master : Get K3s node token (Worker needs this to join)] ***************************************
ok: [master_node]

TASK [k3s_master : Save token as a fact for the worker role to use] ***************************************
ok: [master_node]

TASK [k3s_master : Get Master private IP] ****************************************************************
ok: [master_node]

TASK [k3s_master : Share token and IP with the worker group] *********************************************
changed: [master_node]

TASK [k3s_master : Install Nginx Ingress Controller] *****************************************************
changed: [master_node]

TASK [k3s_master : Get kubeconfig for GitHub Actions] ****************************************************
changed: [master_node]
```

```
PLAY [worker] ********************************************************************************************

TASK [Gathering Facts] ***********************************************************************************
ok: [worker_node]

TASK [k3s_worker : Install K3s worker and join cluster] **************************************************
changed: [worker_node]

TASK [k3s_worker : Wait for K3s agent to start] **********************************************************
ok: [worker_node]

PLAY RECAP ***********************************************************************************************
master_node                : ok=17   changed=10   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
worker_node                : ok=11   changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Cela va créer le fichier l3s-kubconfig.yaml

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ ls -la
total 28
drwxr-xr-x 3 benjo benjo 4096 Mar 25 02:06 .
drwxr-xr-x 9 benjo benjo 4096 Mar 24 22:53 ..
-rw-r--r-- 1 benjo benjo  135 Mar 24 22:53 ansible.cfg
-rwxr-xr-x 1 benjo benjo  227 Mar 25 01:55 inventory.ini
-rw-r--r-- 1 benjo benjo 2941 Mar 25 02:06 k3s-kubeconfig.yaml
-rw-r--r-- 1 benjo benjo  335 Mar 24 22:53 playbook.yml
drwxr-xr-x 5 benjo benjo 4096 Mar 24 22:53 roles
```

Étape 5 : Préparation du pilotage Kubernetes

Récupérez le fichier de configuration généré par Ansible :

1. Modifiez l'IP locale par l'IP publique du Master : sed -i 's/127.0.0.1/PUB_IP_MASTER/' k3s-kubeconfig.yaml

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ cat k3s-kubeconfig.yaml
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUJkekNDQVIyZ0F3SUJBZ0lCQURBS0JnZ3Foa2pPUFFRREFqQWpN
QmhyTTNNdGMyVnkKZG1WeUxXTmhRREUzTnpRek9UY3hOemd3SGhjTk1qWXdNekkxTURBd05qRTRXaGNOTXpZd016SXlNREF3TmpFNApXakFqTVNFd0h3WURWUVFE
ZG1WeUxXTmhRREUzTnpRek9UY3hOemd3V1RBVEJnY3Foa2pPClBRSUJCZ2dxaGtqT1BRTUJCd05DQUFSd3VlWWJSb2dnYmdmWmVTdll2OFFxUHBkRXNqRjNqZDJu
L3BTQVY0UkVhdkVEMlQwOG03b0t5UEFRWTRnRGFQRDNpM2wyemNSWW5vMEl3UURBT0JnTlZIUThCQWY4RQpCQU1DQXFRd0R3WURWUjBUQVFIL0JBVXdBd0VCL3pB
VUhyRjlRbmxnZmJGVVVOMWkvMXFiCmRSRW42WU13Q2dZSUtvWkl6ajBFQXdJRFNBQXdSUUloQU5XNFAyWjlFQ29VYmJsSnZuLzJJSm84bXhGa0FVRVIKd0JZU2NN
SHpZVjZTVU5UTXg5K3dpdVE1MWh0RzNZRU84eEhUc2UxaTFSWTB3UT09Ci0tLS0tRU5EIENFUlRJRklDQVRFLS0tLS0K
    server: https://127.0.0.1:6443
  name: default
contexts:
- context:
    cluster: default
    user: default
  name: default
current-context: default
kind: Config
users:
- name: default
  user:
```

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ sed -i 's/127.0.0.1/158.158.43.112/' k3s-kubeconfig.yaml
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$
```

2. Testez l'accès : export KUBECONFIG=./k3s-kubeconfig.yaml && kubectl get nodes

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ ansible master -m fetch -a "src=/etc/rancher/k3s/k3s.yaml dest=./k3s-kubeconfig.yaml flat=yes"
master_node | SUCCESS => {
    "changed": false,
    "checksum": "079e97f4b0c0afced9ef3238e0c29ad49671a3b4",
    "dest": "/home/benjo/edu-path-ai/ansible/k3s-kubeconfig.yaml",
    "file": "/etc/rancher/k3s/k3s.yaml",
    "md5sum": "7147f5e940edc8a23906a0aae1274150"
}
```

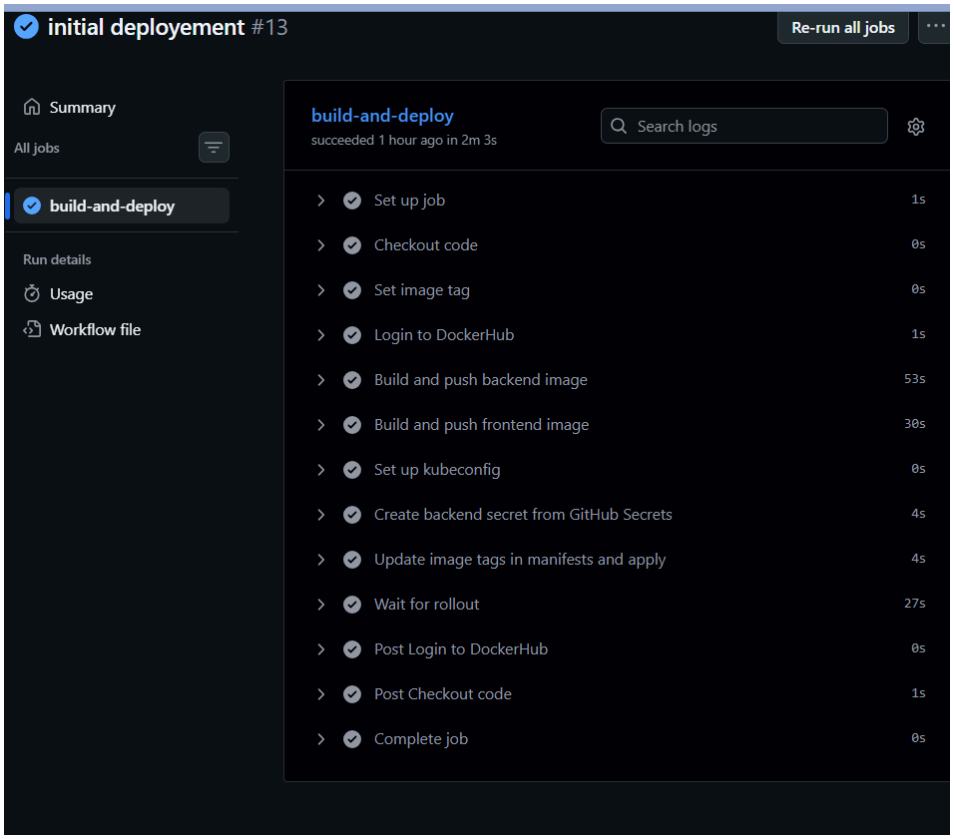
```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ export KUBECONFIG=./k3s-kubeconfig.yaml
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ kubectl get nodes
NAME          STATUS   ROLES           AGE     VERSION
k3s-master    Ready    control-plane   5m29s   v1.34.5+k3s1
k3s-worker    Ready    <none>          5m9s    v1.34.5+k3s1
```

8.3 Initialisation du Pipeline CI/CD (GitHub)

Pour activer le déploiement automatique, configurez les **Secrets** dans votre dépôt GitHub (*Settings > Secrets and variables > Actions*) :

- **DOCKERHUB_USERNAME / TOKEN** : Vos accès Docker Hub.
- **MASTER_PUBLIC_IP** : L'IP de Node Master Azure.
- **KUBECONFIG** : Le contenu complet du fichier k3s-kubeconfig.yaml.
- **JWT_SECRET_KEY** : Une clé aléatoire pour sécuriser l'API Backend.

Une fois les secrets enregistrés, effectuez un git push origin main pour déclencher le premier déploiement automatique.



8.4 Maintenance et Destruction

- **Vérification :** utilisez kubectl get pods pour vérifier que les services Frontend et Backend sont bien Running.

```
ansible.cfg  inventory.ini  k3s-kubeconfig.yaml  playbook.yml
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
backend-6c988dc4f4-mk4m7    1/1     Running   0          79s
frontend-7697886c7-4rmlv    1/1     Running   0          78s
```

- **Accès :** Récupérer l'IP externe de l'Ingress pour ouvrir l'application dans le navigateur.

```
benjo@DESKTOP-4GCHLHE:~/edu-path-ai/ansible$ kubectl get service -n ingress-nginx
NAME                                 TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)                      AGE
ingress-nginx-controller             LoadBalancer   10.43.50.50    10.0.1.4      80:31846/TCP,443:32551/TCP   31m
ingress-nginx-controller-admission   ClusterIP      10.43.44.254   <none>        443/TCP                      31m
```

- **Nettoyage :** Pour éviter de consommer les crédits Azure inutilement on peut détruir l'intégralité des ressources avec : terraform destroy -auto-approve

IX. Conclusion

9.1 Bilan Technique et Défis Surmontés

Ce projet a permis de mettre en place une chaîne de valeur **DevOps** complète, allant du code source d'un modèle de Machine Learning jusqu'à son déploiement orchestré sur un cluster Kubernetes en production.

- **Réussite de l'automatisation :** L'objectif "Zero-Touch" a été atteint. Grâce au couplage de **Terraform** et **Ansible**, le temps de mise en place d'une infrastructure complexe a été réduit de plusieurs heures à quelques minutes, tout en éliminant les erreurs de configuration manuelle.
- **Défis du TLS et de l'accès distant :** L'un des obstacles majeurs a été la sécurisation de l'API K3s. La gestion du paramètre --tls-san a été indispensable pour permettre à GitHub Actions de piloter le cluster à distance sans compromettre la sécurité des certificats.
- **Contraintes de ressources :** Travailler avec des quotas Azure limités et 4 Go de RAM par nœud a imposé une rigueur extrême dans l'optimisation des images Docker (multi-stage builds) et le choix d'un orchestrateur léger comme K3s.

9.2 Perspectives et Évolutions

Bien que fonctionnelle et robuste, cette infrastructure est conçue comme une base évolutive capable de supporter des charges plus importantes.

- **Scalabilité Horizontale :** Grâce à l'architecture modulaire de Terraform, l'ajout de nouveaux nœuds *Workers* peut se faire en modifiant une simple variable, permettant au cluster de supporter une montée en charge des utilisateurs.
- **Sécurisation HTTPS (Cert-Manager) :** Une évolution naturelle serait l'intégration de **Cert-Manager** avec Let's Encrypt. Cela permettrait d'automatiser la génération et le renouvellement de certificats SSL, offrant une navigation sécurisée en HTTPS sans intervention manuelle.
- **Monitoring Avancé :** L'implémentation d'une stack **Prometheus & Grafana** permettrait d'obtenir une visibilité en temps réel sur la consommation des ressources des Pods et les performances du modèle de recommandation, facilitant ainsi la maintenance préventive